

CITS3011 Intelligent Agent Project Report

Lewei Xu (23709058)

Basic Techniques

1. Internal State Representation

This was the first technique/idea that I implemented that is the foundation of all the other techniques described below. Inspired by the internal state graph implemented in **GreedyAgent**, I adapted this graph that uses the **networkx** library to include various other information in its nodes and edges. The implementation of this internal state representation is contained within the lines 36-85 in **agent_23709058.py**, and contains 2 functions: **build_state_graph()** and **update_state_graph()**.

build_state_graph()

- This function is used to construct the internal state representation of the diplomacy game, and is called once within **new_game()**.
- We first construct the nodes of the graph using **game.map.loc_type** which is a dictionary of locations and the locations type. With this information, we can initialize each node as a location, with 2 attributes **type** which corresponds to the unit type (WATER, LAND or COAST) and **units** which is a 2-tuple storing the type of unit (A or F) and the power that unit belongs to.
- Next we use **game.map.abuts()** method to construct the edges between the nodes. For each pair of nodes in the state graph, we test if an army and a fleet can move between the nodes, and if it can, we add the edge and set the corresponding attribute **army** and/or **fleet** to **True**. For example, to test if an army can move between 2 nodes:

```
if self.game.map.abuts('A', i, '-', j):  
    state_graph.add_edge(i, j, army=True, fleet=False)
```

update_state_graph()

- This function is used to update the internal state representation every phase, and is called within **update_game()** after the game engine processes the new moves.
- First, we initialize (and reset) a couple more attributes for each node, these include **is_supply** which is a boolean indicating whether a node is a supply center, and **owner** which is a string containing which power currently owns the supply center (if it is one).
- Finally, we use the diplomacy library's game object to find which locations are supply centers, where each unit is located, and who owns each supply center, then update the internal state representation with this information.

This internal state representation of the game is crucial for the other techniques implemented, as it allows for conventional BFS, DFS and other graph traversal algorithms to be directly used for choosing the orders that are to be given to each unit. Although all this information is contained within the diplomacy game object, it is not easily usable, and we would need to write custom algorithms to find information such as shortest paths between 2 locations.

At this stage, there is no real experimental data regarding agent performance to be provided, however, previously when directly using the game object to get information regarding the current phase, it was found that there were no real demerits in terms of time complexity when constructing and constantly updating an internal state representation. This is due to the fact that all operations run in $O(n)$ time anyway, so constructing an internal state representation will only serve to make other techniques more easy to implement.

2. Case-Based Reasoning

After playing webdiplomacy against bots for some time, I designed a heuristic that describes a number of cases that a good player would generally make. These cases utilize the internal state representation described

above to work cleanly. The implementation is contained with the `get_actions()` method. Initially there were only 4 cases:

- Case 1: for every unit that is currently in a supply center that is not yet owned by us, hold. This generally applies to when a unit has moved into an unoccupied supply center in the spring phase, and we do not yet own the supply center, so we would hold and wait for the fall phase so that we can build more units during build phase.
- Case 2: for remaining units, check neighboring nodes for unoccupied supply centers and move there. Building more units should always be a priority, and to build more units, we need to capture more supply centers.
- Case 3: for remaining units, if >2 units are adjacent to a location occupied by an enemy unit, move one unit to that location and have all other units support the move/attack. This case is mainly to simulate an aggressive agent that will always try to take space if it can.
- Case 4: Move all remaining units that haven't yet been given an order closer to their closest supply center not owned by us via the shortest path. This is so that there is a higher chance for Case 2 and Case 3 triggering in the next phase.

The above cases outlines how the agent works at a high level. Later on, more cases were added to fine tune the behavior of the agent:

- Case 0: if in a build phase, build a random unit in a home supply center
- Case 0.5: if in a retreat phase, retreat to a random adjacent location if possible, else disband
- Case 2.5: an extension of Case 2 that also checks for convoy options
- Case 3.5: move any fleets into a WATER location if possible to vacate space for army units and also allow for convoys

This case-based reasoning is mean to simulate a relatively beginner human playing diplomacy. Due to the nature of Scenario 1, I went with this approach to achieve a 100% win rate in Scenario 1 where all other agents are static. At the same time, as this agent is versing other agents, the relatively aggressive heuristic should also work quite well against other simple agents.

With just cases 1-4 + case 0 that allows for building more units, the agent was able to achieve a relatively high win rate in scenario 1 of $>70\%$. However, when implementing case 0.5, 2.5 and 3.5 which was added after visualizing a couple games where the agent failed to win, the agent managed to achieve a 100% win rate in scenario 1. Next, I tested this agent in scenario 2 with 20 iterations per power, and found that it actually achieved an approximately 40% win rate with around 12 supply centers captured on average. This shows that the aggressive agent worked very well against other agents.

New Techniques

1. Power-Specific Strategic Variation
2. Dynamic Supply Center Targeting